一种时域卷积注意力机制优化的双向长短期网络
——基于粤港澳气象大数据的中期多站点协同预测与分析

摘要

近年，全球气象系统的不稳定态势愈发明显。准确高效的气象预测作为现代气象业务的核心，是搭建气象防灾减灾体系的重中之重。传统预测技术难以高效应对复杂混沌的大气系统。其中，应用广泛的中期多站点协同预测一直是一项极具挑战性的任务，如何有效挖掘中期时序依赖与多站点间复杂的耦合关系是设计气象预测模型的关键。在气象大数据时代下，利用数据驱动“智脑”输出预测结果的人工智能方法成为攻破此任务的最佳选择。针对中期多站点协同预测这一关键科学问题，本团队提出一种时域卷积注意力机制优化的双向长短期记忆神经网络。首先将通道空间注意力机制（CBAM）嵌入时域卷积神经网络(TCN)，提高网络对气象数据中时间自相关与空间互相关的表征能力，随后通过双向长短期记忆神经网络(BiLSTM)强化混合架构对中期时序依赖的提取效果。在 2017-2023 年粤港澳地区的四类气象要素数据上（涵盖 17 个站点）的实验表明，所提模型比基准模型的 MAE、RMSE 平均降低 30%左右。

关键词：气象大数据、中期多站点协同预测、混合神经网路架构、通道空间注意力机制

目录

摘要.....I
目录.....II
表格与插图清单.....IV
一、 引言.....1
（一） 研究背景及意义.....1
（二） 气象预测研究现状与研究内容.....3
1. 研究现状.....3
2. 基于物理方法的数值天气预报.....3
3. 统计方法与传统机器学习方法.....4
4. 基于深度学习的人工智能方法.....5
① 传统的人工智能方法.....5
② 注意力机制.....5
5. 研究现状分析与展望.....6
（三） 研究内容与创新性.....6
1. 研究内容.....6
2. 创新与贡献.....7
二、 方法.....8
（一） 时域卷积神经网络.....8
（二） 通道空间注意力机制.....10
（三） 双向长短期记忆神经网络.....12
（四） 基于时域卷积注意力机制优化的双向长短期记忆神经网络.....13
三、 实验.....14
（一） 数据来源及预处理.....14
1. 数据来源.....14

2. 缺失数据处理 15
3. 异常数据处理 17
4. 数据标准化处理 19
5. 实验环境 19
6. 损失函数与模型评价指标 20
7. 对比实验与消融实验 20
① 对比试验 20
② 消融实验 22
四、 结论与探讨 25
（一） 模型的优势 25
（二） 模型的不足及改进 26
五、 参考文献 27
六、 附录 28

表格与插图清单

表 1 数据缺失情况示意表.....16
表 2 温度数据集插补后统计量情况.....17
表 3 对比实验结果.....21
表 4 消融实验结果.....23

图 1 研究思路.....7
图 2 扩散因果卷积.....8
图 3 时域卷积块.....9
图 4 通道空间注意力机制.....10
图 5 嵌入通道空间注意力的时域卷积块.....11
图 6 长短期记忆神经网络.....12
图 7 双向长短期记忆神经网络.....13
图 8 所提模型（TCCB）.....14
图 9 粤港澳 17 个气象站示意图.....15
图 10 温度异常值处理.....18
图 11 气压异常值处理.....18

算法 1：汉普尔滤波器算法.....19

一种时域卷积注意力机制优化的双向长短期网络
——基于粤港澳气象大数据的中期多站点协同预测与分析

一、 引言

（一） 研究背景及意义

人类进入工业时代以来，全球气候系统呈现逐年变暖的趋势，极端天气近年频繁出现。根据世界气象组织发布的《2023 年全球气候状况》，2023 成为有记录以来的最热年份，全球沸腾时代悄然开始。极端高温带来的台风暴雨、山洪野火等自然灾害给在过去半个世纪造成 4.3 万亿美元经济损失[1]。着眼国内，2023 年 8 月 28 日台风“海葵”在我国台湾登陆，席卷粤港澳等地区，香港天文台刷新有记录以来最高降雨量；2023 年小麦夏收期间，河南省遭遇十年内最严重的“烂场雨”（持续阴雨天气），受灾面积占全国小麦面积 8%。根据《中国气候公报 2023》报道，2023 年全国气象灾害造成 1053.9 万公顷农作物减产，直接经济损失高达 3306 亿人民币[2]。

针对我国气象灾害增多增强的实际情况，开发准确高效的气象预测模型是保障国民生命健康，护航生产活动稳步开展的重要手段。**随着卫星对地观测技术与气象数据库技术的蓬勃发展，如何有效挖掘海量气象数据的潜在价值，充分利用各类气象数据要素创造的信息增益，进而将瞬息万变的气象状况转为可预知可确定的实时播报，成为构建现代化气象体系的重要一步。**《全国气象发展“十四五”规划》明确指出我国要加快推动人工智能、云计算等新一代信息技术落地气象行业，依托气象大数据云平台等关键核心技术，引领气象现代化建设迈向下一个新台阶[3]。

人工智能预测技术与气象大数据的深度融合正成为气象防灾减灾机制的重

要防线[4]。在短中长期等不同时间尺度的预测技术中，**中期气象预测（72h-240h）对于粮食生产、能源调度、防灾减灾等诸多领域的影响尤为明显。**相较于短期气象预测，中期预测能为相应部门的决策工作提供较为宽裕的准备时间；相较于长期气象预测，中期预测往往精度更高，适用范围更大。**同时，开发多站点协同预测技术是搭建智慧气象大平台的必要手段**，一方面多站点协同预测有助于减少气象预测模型出现数量冗余，质量不佳等情况；另一方面充分发挥气象大数据优势，助力模型捕获站点与站点间的信息特征，进而提高模型预测的精度与准度。然而，此类中期多变量气象数据预测任务给研究工作提出较大挑战，如何厘清不同时间、不同站点间的非线性相互依赖关系是设计相应模型架构的关键环节。**因此，设计一种能有效捕捉气象数据的时间自相关性与站点互相关性的人工智能方法极具现实意义。**

粤港澳城市群地理区域近海、人口密度集中、经济水平高等特性导致其成为典型的气候灾害频发区，探究准确有效的气象预报技术有助于提高粤港澳地区对抗自然灾害的能力。与此同时，粤港澳地区出色的气象数据平台建设，为探究基于数据驱动的人工智能方法提供丰富数据资源。并且基于粤港澳地区的气象预测技术可为全国其他地区提供丰富的理论参考与实际价值。

在此背景下，本团队将基于 2017 年-2023 年粤港澳 17 个气象站点的温度、露点温度、气压、风速四类气象要素数据，开发一种用于中期多变量气象数据预测任务的新型神经网络架构。该架构针对多变量、中期时序依赖等关键问题，有机融合了通道空间注意力机制、时域卷积神经网络、双向长短期记忆神经网络等人工智能技术。**本研究的完成将为气象预测领域提供一种新方法新视角；为诸多预测场景提供一种实用性有效性的预测工具；从而在气象大数据时代下，为人工**

智能理论与应用工作添砖加瓦。

（二） 气象预测研究现状与研究内容

1. 研究现状

气象预测是一项依赖可靠数据，用于对温度、风速、露点温度等大气状况进行准确预测的科学活动[5]。随着计算机技术和卫星遥感技术的不断发展，研究人员利用强大的算力支持和海量的气象数据，不断提高预测结果的精度与准度。**截至目前，气象预测理论与技术可分为三类方法：基于物理方法的数值天气预报（NWP）、统计方法与传统机器学习方法以及基于深度学习的人工智能方法。**

2. 基于物理方法的数值天气预报

20 世纪初，Abbe（1901）[6]和 Bjerknes（1904）[7]提出大气预测可被视为一种数学物理方法，通过求解一组基于大气物理学、流体力学、热力学等学科的非线性微分方程，可得出下个时间步的大气状态。近年，许多研究人员开发了优秀的数值天气预报系统。例如，Wang H 等（2022）[8]将中国化学天气预报系统在线集成到中尺度数值天气预报，建立了化学-天气交互模型，并在中国 6 个主要污染次区域进行模式性能测试，结果出色。数值天气预报系统在处理地面风速预估时，不确定性的地面效应给预测工作造成巨大困难，Feng J 等（2022）[9]通过在离线地表多层模型中添加优化函数，纠正地面效应带来的误差，并与其他修正方法对比，性能提升明显。**数值天气预报系统以其精度高等优点，深受研究人员喜爱。但是求解偏微分方程组在计算机技术高度发达的当代，仍然颇具挑战。2023 年，**

华为公司的盘古大模型在速度与精度均超过当前最强大的数值天气预测系统，人工智能成为比数值天气预报更具研究前景的选择。

3. 统计方法与传统机器学习方法

统计方法是一种利用气象数据的时序性、自相关性等统计特性建立简单数学模型的方法[10]。凭借可解释性强、计算成本低等优点，统计方法被广泛应用。Farsi M 等（2021）[11]利用遗传算法优化季节性自回归滑动平均模型，解决季节性自回归滑动平均模型求最优解的困难，实验结果表明所提方法相较于基准模型更优秀。**此类基于统计方法的模型不仅需要数据质量良好（满足平稳性、服从正态分布等），并且难以处理非线性序列数据。因此，仅依靠统计方法难以捕捉现实世界中混沌的气象数据信息。**传统机器学习方法在刻画非线性数据信息中表现出色，研究人员对此开展了广泛研究。尹彤（2022）[12]利用随机森林、梯度提升机和 XGBoost 等机器学习模型对武汉市空气质量指数进行预测，并对上述模型进行 stacking 模型融合，实验表明随机森林和 XGBoost 组合效果最佳。孟建军等（2024）[13]利用改进的秃鹰搜索算法对 XGBoost 进行初始参数优化，并通过结合除风速外的气象特征。实验结果表明，所提模型相较于其他基准模型，在多步预测的精度和智能算法的寻优效果上均提升明显。**但是传统机器学习方法十分依赖特征工程，特征工程的好坏直接影响预测的效果。气象系统的混沌性导致特征工程的实施异常困难，并且机器学习处理海量数据方面仍存在缺陷。因此，具备从海量数据中自动捕捉有价值特征的深度学习方法，能有效弥补传统机器学习的不足[14]。目前，研究高效强大的神经网络框架成为气象预测领域最具活力的方向。**

4. 基于深度学习的人工智能方法

① 传统的人工智能方法

深度学习方法是一种依靠人工神经网络，进行表征学习的算法。常见的神经网络架构包括循环神经网络（RNN）和卷积神经网络（CNN）等，循环神经网络以其出色的序列数据处理能力成为时间序列预测任务最常用的模型。然而对于长期的预测任务，循环神经网络极易出现“遗忘”情况，其变体长短期记忆网络和门控循环单元则具备长期记忆的功能。近年，许多研究人员对深度学习在气象预测领域的应用展现浓厚的兴趣。Zang H 等（2020）[15]同时利用卷积神经网络和循环神经网络处理太阳辐射数据的时空特征，通过卷积神经网络提取多站点间的气象空间特征，然后利用长短期记忆网络提取序列数据中的时间特征。所提方法相较于其他模型，在三个数据集上具有一定优势。Karevan Z 等（2020）[16]提出一种转换式长短期记忆神经网络，通过利用局部信息进行时间序列预测。在多个气象数据集上，作者验证该方案的有效性。

② 注意力机制

注意力机制起源于计算机视觉领域，其核心思想是让模型注意力从全局特征转向局部重点。Vaswani A 等（2017）[17]基于注意力机制，提出一种全新的神经网络框架 Transformer。后续许多研究人员在此基础上进行改进，提出各类针对时序任务的 Transformer 类模型。Zhou H 等（2021）[18]结合 ProbSparse 自注意力机制、自注意力蒸馏、生成式解码器，提出能有效处理极长序列数据的 Informer。Wu H 等（2023）[19]在 Autoformer[20]的基础上提出了具有多重相关机制的

Corrformer，该模型将全球气象站的空间互相关性和时间自相关性统一为可学习的多尺度树结构，实现基于多达数万个气象站点的协同气象预报，并成功完成2022年冬季奥运会的天气预报任务。

5. 研究现状分析与展望

综上所述，在数值天气预报系统（NWP）庞大的计算成本（盘古为代表的人工智能技术在精度不减的情况下，速度是NWP的10000倍）与统计方法和传统机器学习方法难以有效刻画复杂的气象数据特征的当下，**基于深度学习的人工智能方法以其强大的表征能力，成为气象预测的领域不二选择。针对单一的CNN、RNN 神经网络难以准确捕捉多变量和多时间步间的耦合关系[21]，本文将通过引入一种注意力机制，搭建混合神经网络架构，进而提高气象预测的精度。**

（三） 研究内容与创新性

1. 研究内容

针对中期多元气象序列数据预测任务，本文主要提出一种基于时域卷积神经网络（TCN），通道空间注意力机制（CBAM），双向长短期记忆神经网络（BiLSTM）的混合神经网络架构CBAM-TCN-BiLSTM（CBTB）。**结合通道空间注意力机制的时域卷积神经网络（CBTCN）在保留TCN的优势下，进一步对多变量（通道）、多时间步（空间）进行注意力加权，使模型能更专注于重要特征的提取；双向长短期记忆神经网络则能从提取后的特征中，利用上下文信息进一步捕捉气象时序数据的中长期依赖关系。**研究思路如图 1所示。

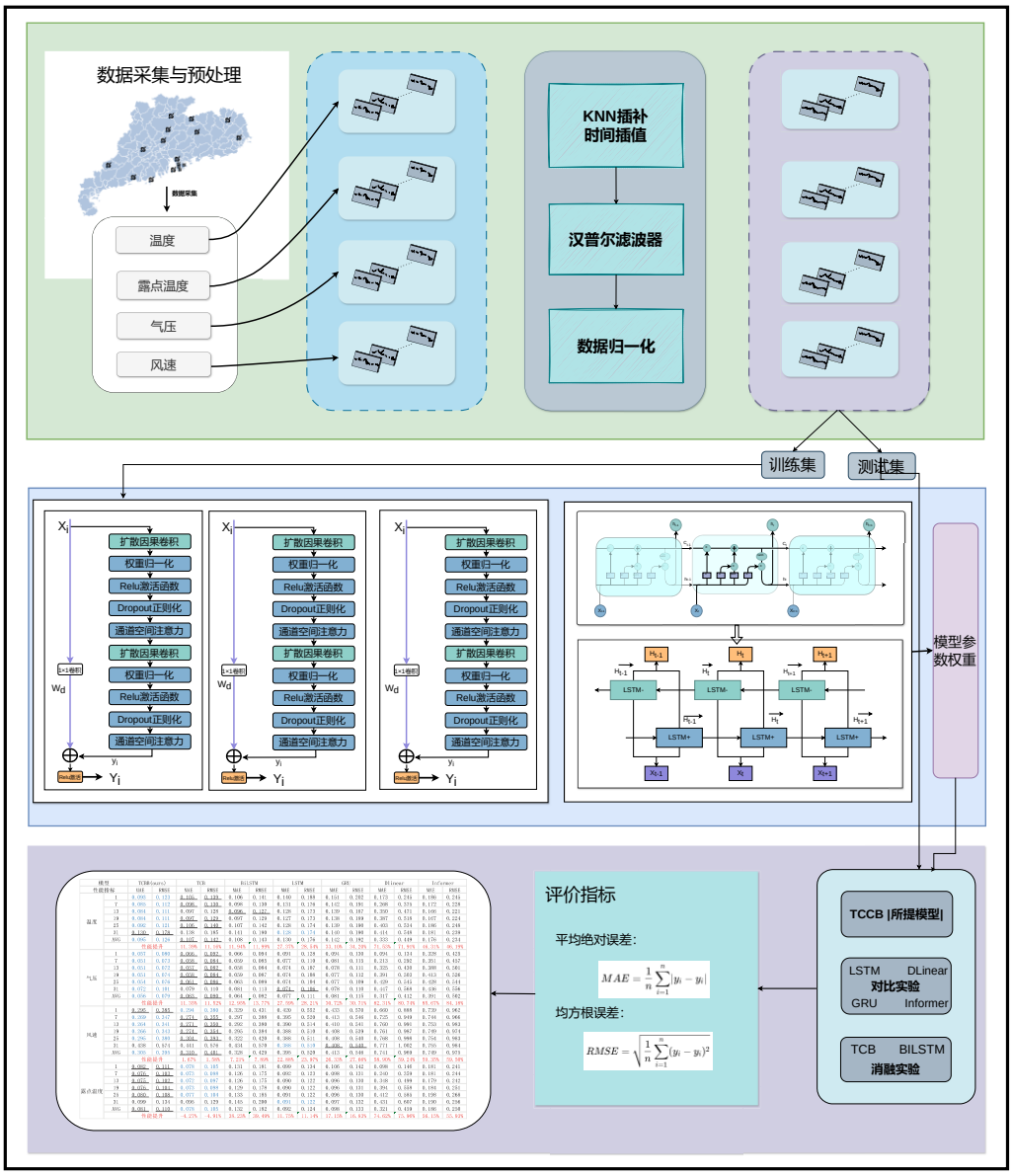


图 1 研究思路

2. 创新与贡献

①本研究系统收集与整理了 2017 年-2023 年粤港澳地区气象数据，使其能通过简单的处理工作即可部署在基于数据驱动的预测模型中。

②本研究针对中期多元气象序列预测任务中，将具备提取复杂特征的时域卷积神经网络、有效捕捉多变量与多时间步中重要特征的通道空间注意力机制、结

合上下文信息提取序列数据长期依赖的双向长短期记忆神经网络进行有机融合，提出一种基于时域卷积注意力机制优化的双向长短期记忆神经网络，并通过气象数据集证明所提模型的性能优异。

③本研究将在附录提供所提模型的代码。该代码基于拥有强大社区支持的 pytorch 框架编写，并附上详细注释。相关工作者可以十分轻松地在此基础上进行广泛的应用或改进。

二、 方法

（一） 时域卷积神经网络

Bai S 等（2018）[22]成功将卷积架构引入序列数据建模领域，提出时域卷积神经网络（TCN），打破循环架构在序列数据建模领域“一家独大”的场面。其核心内容为扩散因果卷积如**图 2** 所示。

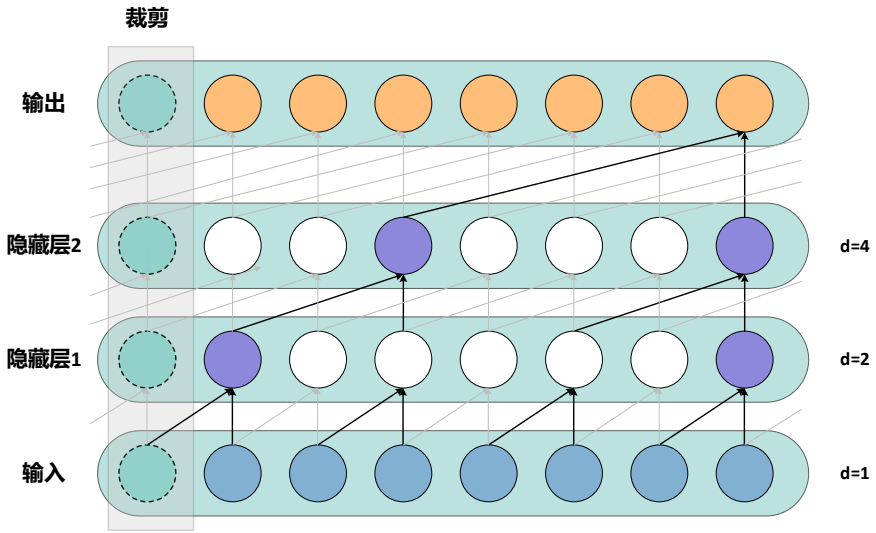


图 2 扩散因果卷积

在保证未来信息的输出仅和过去信息进行卷积运算得到的同时，通过扩散因子和卷积核大小，扩大卷积的感受野，使其在不增加额外参数下捕获更长时间范围的

信息。假设输入序列为$\boldsymbol{x}=\{x_1,x_2,\dots,x_T\}$，卷积核运算输出序列为$\boldsymbol{g}=\{g_1,g_2,\dots,g_T\}$，扩散因果卷积数学表达如式(1)。

$$g_t=\sum_{i=0}^{k-1}f(i)\cdot x_{t-d\cdot i} \tag{1}$$

其中，d代表扩散因子，k代表卷积核大小，$t-d\cdot i$代表时间步$t-d\cdot i$到时间步t，$f(i)$代表卷积核运算。

为解决神经网络深度增加带来的网络退化问题（梯度消失、梯度爆炸），TCN作者引入残差连接[23]，具体表现为将输入和输出进行加合后，通过激活函数增加非线性表示能力，如式(2)。

$$\boldsymbol{Y}=ReLU(\boldsymbol{x}+\boldsymbol{y}) \tag{2}$$

其中$\boldsymbol{x}$为输入，$\boldsymbol{y}$为时域卷积部分的输出，$(ReLU)$为激活函数。

每个时域卷积块如**图 3** 所示。

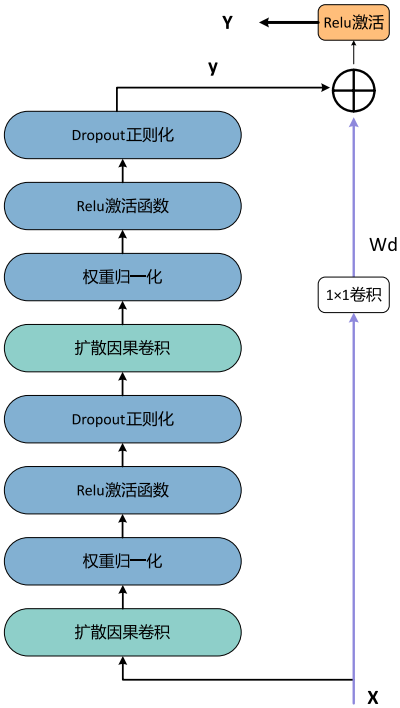


图 3 时域卷积块

通过权重归一化层预防梯度爆炸问题，$ReLU$激活函数和$Dropout$正则化在增加非线性的同时，降低模型过拟合的风险。最后将经过处理的特征进行残差连接和$ReLU$激活处理。

（二） 通道空间注意力机制

对于一个多元时间序列数据而言，并非所有变量对预测的贡献同等重要、同样并非所有输入的时间步信息都存在价值。只有对预测任务重要的特征才值得关心（注意）。通道空间注意力机制起源于计算机视觉领域[24]，通过一定的调整可以和时域卷积进行完美结合。下面将详细阐述该注意力机制两大核心组成：通道注意力、空间注意力，如**图 4** 所示。

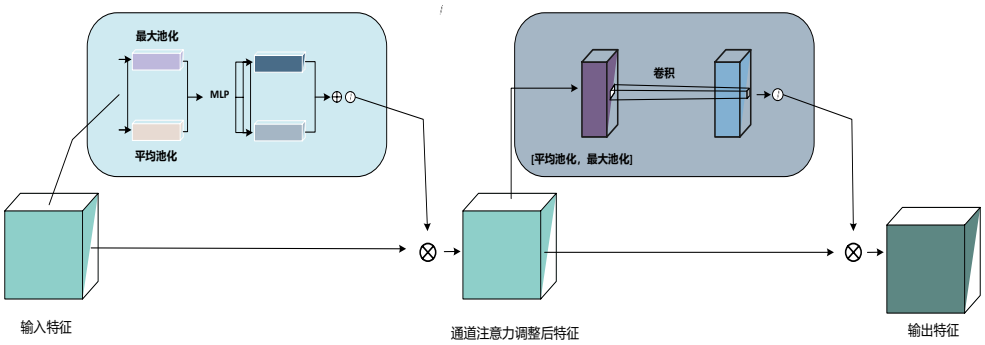


图 4 通道空间注意力机制

调整后的通道注意力机制可以理解为“变量注意力机制”，通道注意力机制首先将输入特征沿时间步维度进行最大池化和平均池化，然后通过多层感知机（MLP）学习重要变量信息，最后对两部分特征进行求和与(sigmoid)激活操作，得到通道注意力权值，如式(3)。

$$M_C(F) = sigmoid\left(MLP\big(AvgPool(F)\big) + MLP\big(MaxPool(F)\big)\right) \tag{3}$$

其中F为输入特征，$AvgPool$为平均池化，$MaxPool$为最大池化，$(sigmoid)$为激

活函数，MLP为多层感知机，$M_C(F)$为通道注意力机制处理后的特征。调整后的空间注意力机制可以理解为“时间步注意力机制”，与通道注意力机制不同的是空间注意力机制在通道维度进行压缩，并通过 7x7 的卷积提取注意力信息，如式(4)。

$$M_S(F_S) = sigmoid\left(\varphi\left(AvgPool(F_S); MaxPool(F_S)\right)\right) \tag{4}$$

其中输入特征$F_S = F \times M_C(F)$，$AvgPool$为平均池化，$MaxPool$为最大池化，$(sigmoid)$为激活函数，$M_S(F)$为通道注意力机制处理后的特征，φ是 7x7 卷积运算。

通道空间注意力结合的时域卷积块如**图 5** 所示，其中通道空间注意力优化的时域卷积，部分构图思路参考文献[25]。

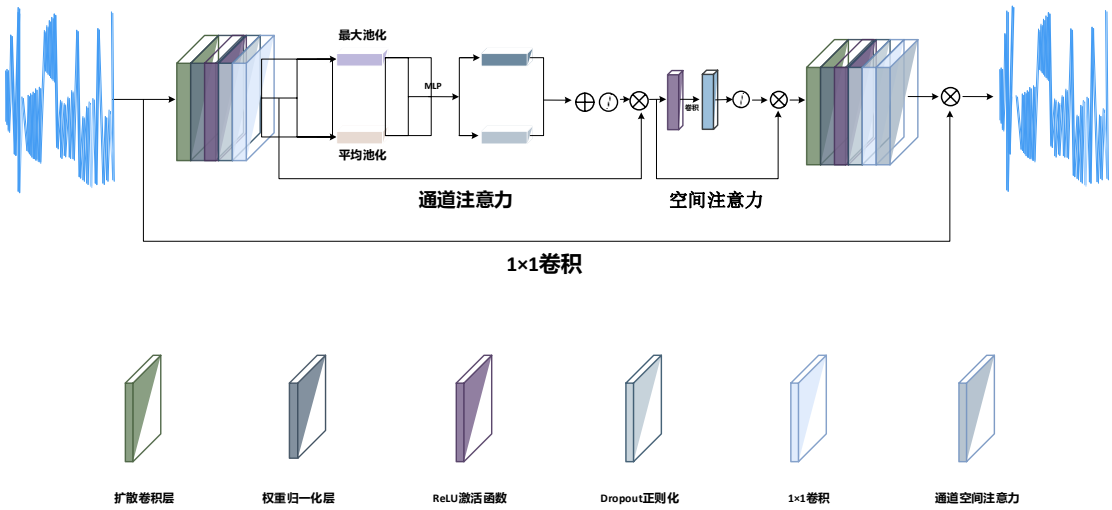


图 5 嵌入通道空间注意力的时域卷积块

具体为在每次卷积操作的正则化处理后面添加通道空间注意力机制。通道空间注意力机制采用通道注意力-空间注意力的串行架构。

（三） 双向长短期记忆神经网络

为解决循环神经网络（RNN）在处理长距离数据时梯度消失的问题，Hochreiter S 等（1997）[26]提出长短期记忆神经网络（LSTM）。Huang Z 等（2015）[27]在此基础上进行改进，提出双向长短期记忆神经网络（BiLSTM）。本段先阐述两者的基础部件 LSTM 单元。每个 LSTM 单元主要由三个门结构（遗忘门、输入门、输出门）组成，如图 6 所示。

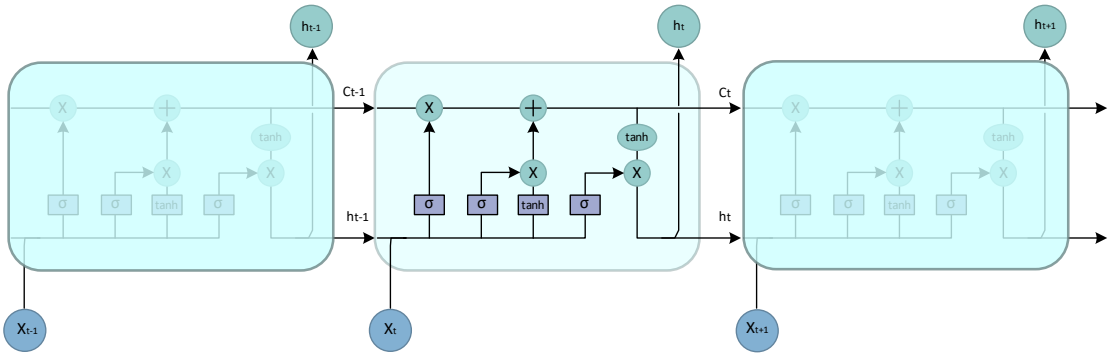


图 6 长短期记忆神经网络

遗忘门的作用是决定当前状态需要遗忘当前那些信息，具体为通过 sigmoid 函数将输入映射到[0,1]之间，0 代表信息应该遗忘，1 代表信息应该保留，如式(5)。

$$f_t = sigmoid\left(W_f \cdot [h_{t-1}, x_t] + b_f\right) \tag{5}$$

其中，W_f代表遗忘门的权重，h_{t-1}代表上一时刻的隐藏状态，x_t代表当前时刻的输入，b_f代表遗忘门的偏置，$(sigmoid)$为激活函数

输入门的作用是得到当前时刻的细胞状态，由两部分组成，首先通过 sigmoid 函数得到$i_t \in [0,1]$，接着将i_t与$\widetilde{C_t} = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$相乘得当前时刻保留的新信息，并对当前保留新信息和过去保留信息进行相加，得到当前保留信息C_t，如式(6)-(7)。

$$i_t = sigmoid(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{6}$$

$$C_t = f_t * C_{t-1} + i_t * tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{7}$$

其中，W_i、W_C分别代表不同操作的权重，h_{t-1}代表上一时刻的隐藏状态，x_t代表当前时刻的输入，b_i、b_C分别代表不同操作的偏置、C_{t-1}、C_t分别代表上一时刻和当前时刻的细胞状态（记忆信息）$(sigmoid)$、$(tanh)$为激活函数。

输出门的作用是输出当前时刻信息，在获得当前时刻的细胞状态（过去与当前对输出有价值的记忆）后，输入经过激活函数过滤的信息，如式(8)。

$$h_t = sigmoid(W_o[h_{t-1}, x_t] + b_o) * tanh(C_t) \tag{8}$$

其中W_o代表权重，h_{t-1}代表上一时刻的隐藏状态，C_t代表当前时刻的细胞状态，x_t代表当前时刻的输入，b_o代表偏置，$(sigmoid)$、$(tanh)$为激活函数。

双向长短期记忆神经网络由正向 LSTM 和反向 LSTM 组成，如**图 7** 所示。

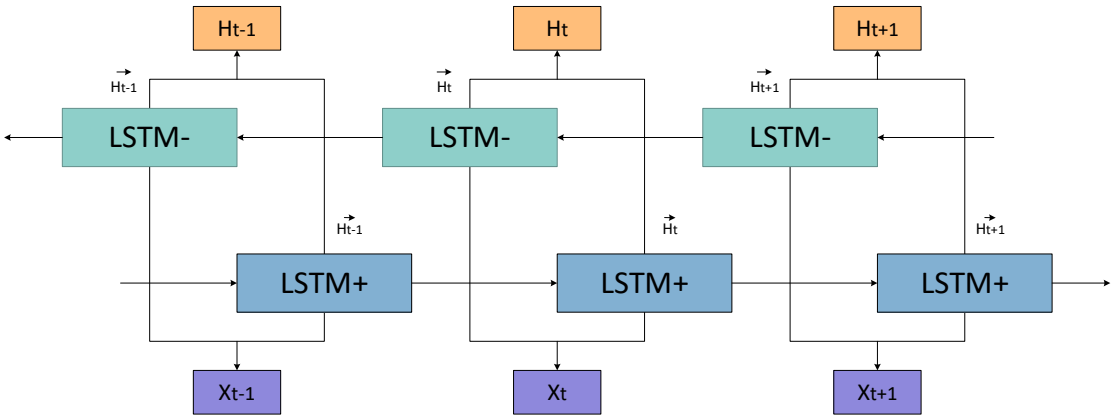


图 7 双向长短期记忆神经网络

通过充分利用序列的正向信息和反向信息，BiLSTM 拥有相较于 LSTM 更强大的捕获中长期依赖关系能力[28]（后续实验同样证明这一点）。

（四） 基于时域卷积注意力机制优化的双向长短期记忆神经网络

所提模型有机结合上述三大核心部件，如**图 8** 所示。

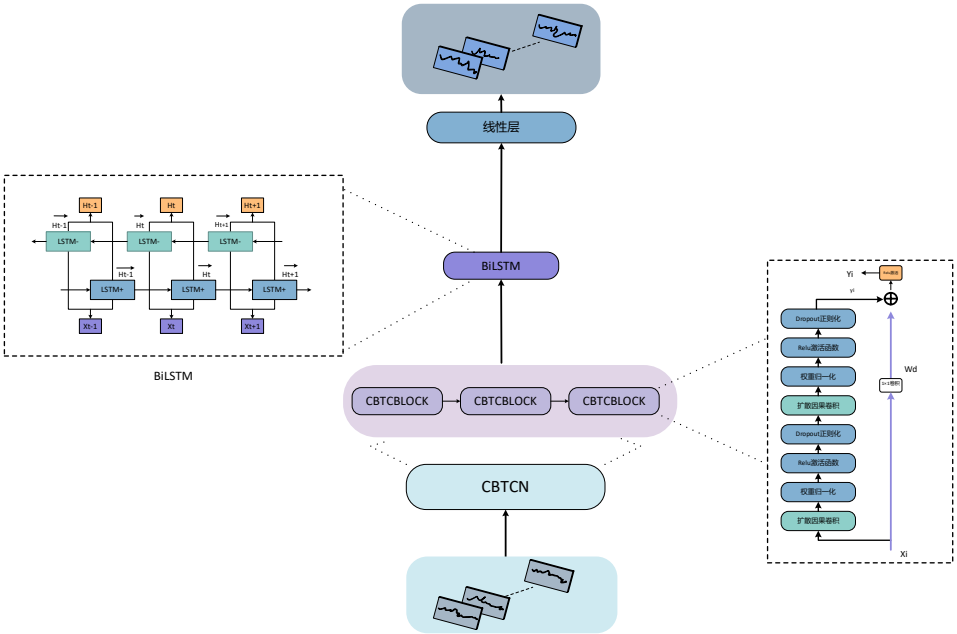


图 8 所提模型（TCCB）

对输入的多元序列数据，首先采用结合通道空间注意力机制的时域卷积神经网络进行特征提取，其组成部分由三个注意力优化的时域卷积块构成。随后利用双向长短期记忆神经网络对处理后的特征进行中长期时序依赖关系捕捉。最后利用线性层调整特征维度，实现基于数据驱动的多对多气象序列预测任务。

三、 实验

本节将详细阐释实验过程的相关细节，**包括数据来源、数据预处理、实验环境、对比实验和消融实验结果等。**

（一） 数据来源及预处理

1. 数据来源

本研究所用数据均通过美国国家气候数据中心收集而得

(https://ngdc.noaa.gov/)，包含粤港澳共 17 个气象站，如**图 9** 所示。

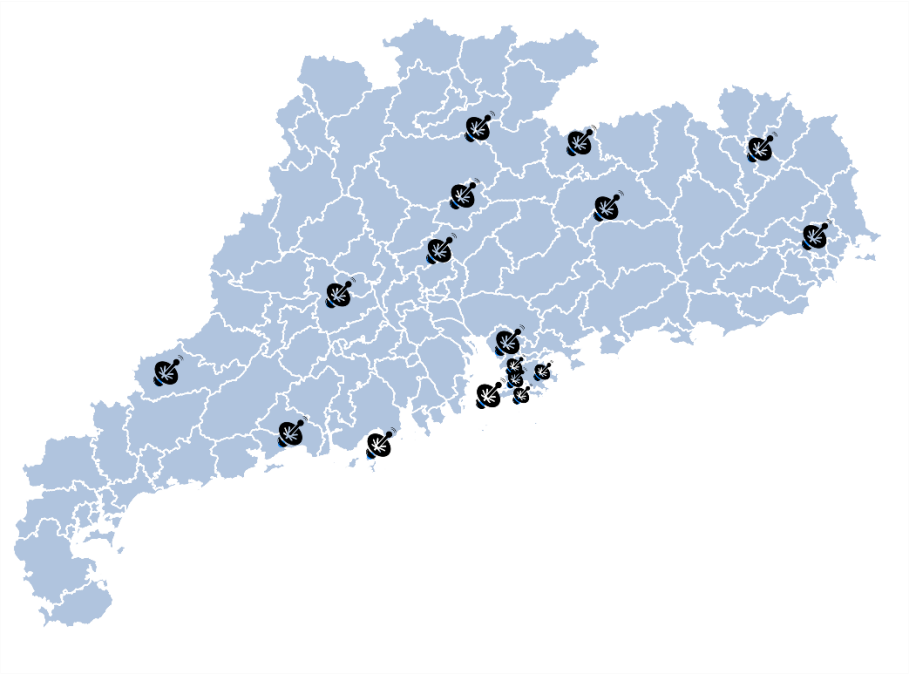


图 9 粤港澳 17 个气象站示意图

每个气象站数据囊括温度、露点温度、气压、风速四类气象因子，收集时间跨度为 2017 年至 2023 年，样本量为 22448，分辨率为 3h。本研究严格按照美国国家气候数据中心提供的说明文档进行数据换算调整，以便后续工作。

2. 缺失数据处理

本研究采用 K 最近邻插补法和按时间插值法结合的方式进行不完整数据处理。根据表 1 可知，缺失数据最高可达 24.31%，直接删除缺失数据将导致多元时间序列可用样本量急剧下降，简单的插值方法（如均值插值、众数插值）在缺失情况严重的数据集并不可取。K 近邻算法作为一种理论成熟的机器学习算法，因其准确率高等优点，广泛应用于数据插补工作。为此，本研究首先利用 K 近邻插补法进行首次插补，具体为通过计算每个样本间的距离，如式(9)，选取最近

的 K 个样本的均值作为插补结果，如式(10)。针对存在大量连续缺失数据，KNN 插补易造成插补结果相同的情况，为此，本研究对 KNN 插补后的数据采用时间插值，如式(11)，替代插补结果相同的样本。插补结果如**表 2** 所示。

$$d(x_i, x_j) = \sqrt{\sum_{l=1}^{m=17} (x_{il} - x_{jl})^2} \tag{9}$$

其中$\boldsymbol{x_i}$、$\boldsymbol{x_j}$为样本向量，x_{il}、x_{jl}为每个站点在对应时刻的值。

$$\widehat{x_{ij}} = \frac{1}{k} \sum_{l \in N_k(i)} x_{lj} \tag{10}$$

其中$\widehat{x_{ij}}$为插补结果，x_{lj}为 K 最近邻特征值。

$$y = y_1 + \frac{(y_2 - y_1)}{(t_2 - t_1)} \times (t - t_1) \tag{11}$$

其中(t_1, y_1)、(t_2, y_2)为已知点，$t_1 < t < t_2$。

表 1 数据缺失情况示意表

数据集\站点	信宜	阳江	肇庆高要	上川岛	韶关	清远佛冈	白云	宝安	香港长洲岛
温度	1.72%	1.73%	1.71%	1.70%	1.73%	14.66%	0.46%	0.50%	1.65%
露点温度	1.72%	1.73%	2.15%	1.71%	1.97%	14.70%	0.46%	0.50%	3.11%
气压	1.76%	1.74%	1.73%	1.72%	1.73%	14.66%	1.72%	24.31%	1.67%
风速	3.76%	1.91%	3.03%	2.51%	1.83%	17.28%	0.46%	0.51%	1.94%
数据集\站点	**香港岛**	**流浮山**	**沙田**	**河源**	**河源连平**	**梅县**	**汕头**	**澳门**	
温度	0.41%	1.98%	1.16%	1.75%	1.78%	1.66%	0.47%	0.53%	
露点温度	0.41%	2.86%	1.39%	1.77%	1.98%	1.66%	0.47%	0.53%	
气压	0.52%	2.53%	1.16%	1.75%	1.79%	1.66%	1.77%	2.90%	
风速	0.41%	2.24%	1.26%	3.97%	3.50%	2.33%	0.48%	0.53%	

表 2 温度数据集插补后统计量情况

统计量\站点	信宜	阳江	肇庆高要	上川岛	韶关	清远佛冈	白云	宝安	香港长洲岛
均值	23.33	23.43	23.02	23.74	21.04	22.10	22.66	23.90	23.27
方差	5.79	5.40	6.32	5.29	7.73	6.68	6.59	5.49	5.11
25%分位数	19.70	19.70	18.60	19.80	15.00	17.30	18.10	20.00	19.50
50%分位数	24.50	24.50	24.20	24.61	22.20	23.20	23.90	25.00	24.00
75%分位数	27.20	27.70	27.50	28.10	26.80	26.90	27.40	28.20	27.10
统计量\站点	香港岛	流浮山	沙田	河源	河源连平	梅县	汕头	澳门	
均值	24.76	23.51	23.99	22.78	20.88	22.40	23.72	23.25	
方差	5.38	5.53	5.50	6.86	7.20	6.98	5.92	5.34	
25%分位数	20.80	19.60	20.00	17.90	15.70	17.50	18.90	19.20	
50%分位数	25.70	24.50	24.90	24.10	22.13	23.60	24.50	24.00	
75%分位数	29.20	27.70	28.50	27.70	26.00	27.30	28.60	27.60	

3. 异常数据处理

气象观测站大多建设在野外，传感器故障、数据传输有误等诸多问题将导致气象观测站收集到不正常数据，这对于预估未来时刻的气象情况造成误导影响。因此，气象序列预测任务中，提前进行异常数据处理是一项常见且重要的手段。常见的异常值检测方法包括 3σ法则、箱体法等，然而此类方法存在一定问题：如要求数据服从高斯分布、不适用于大数据集、并未考虑数据的时序特征等，同时此类检测方法无法对异常数据进行处理。对此，本研究采用汉普尔(hampel)滤波器，该方法是一种异常值检测和去除的滤波方法，通过数据点与移动窗口内的中位数之间的偏差识别异常值，并用中位数替代异常值，具体如下**算法 1** 所示。由于其对于异常值的识别鲁棒性强、不受数据分布影响、采用移动窗口识别时间特征等优点，汉普尔滤波器具有准确稳健识别与处理异常时序数据的强大能力。为方便展示异常值处理情况，下述将以温度、气压数据集的部分站点作为代表展示成果，如下**图 10**、**图 11** 所示。

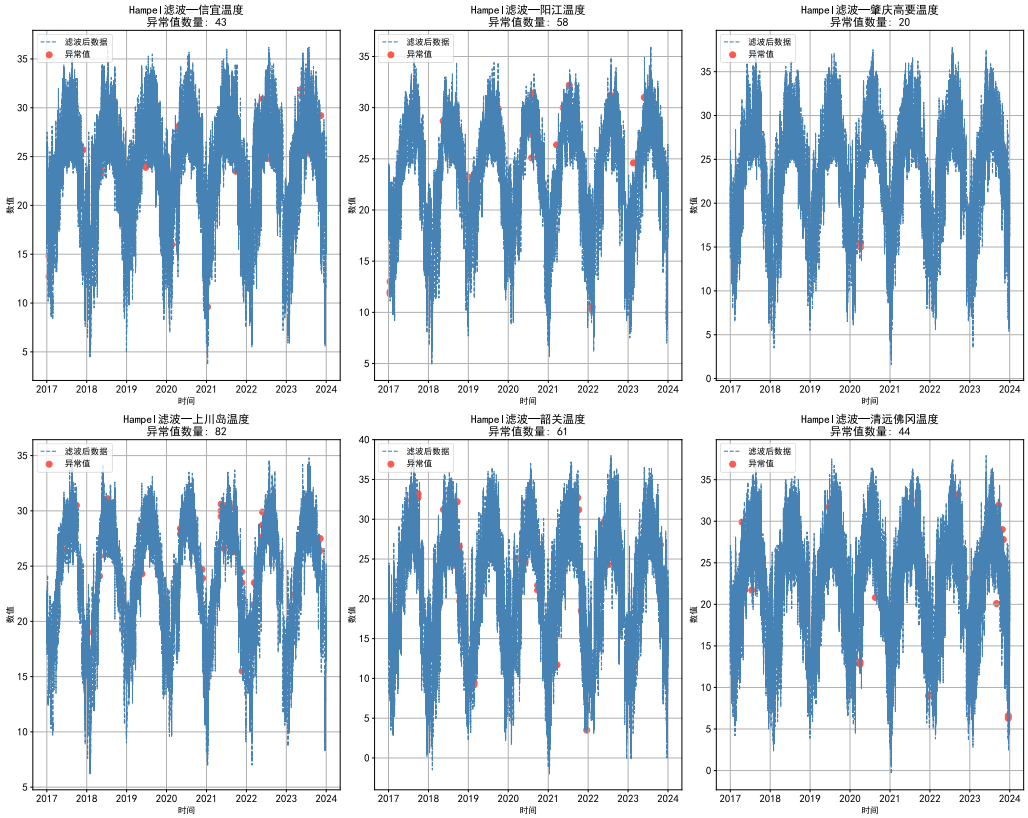


图 10 温度异常值处理

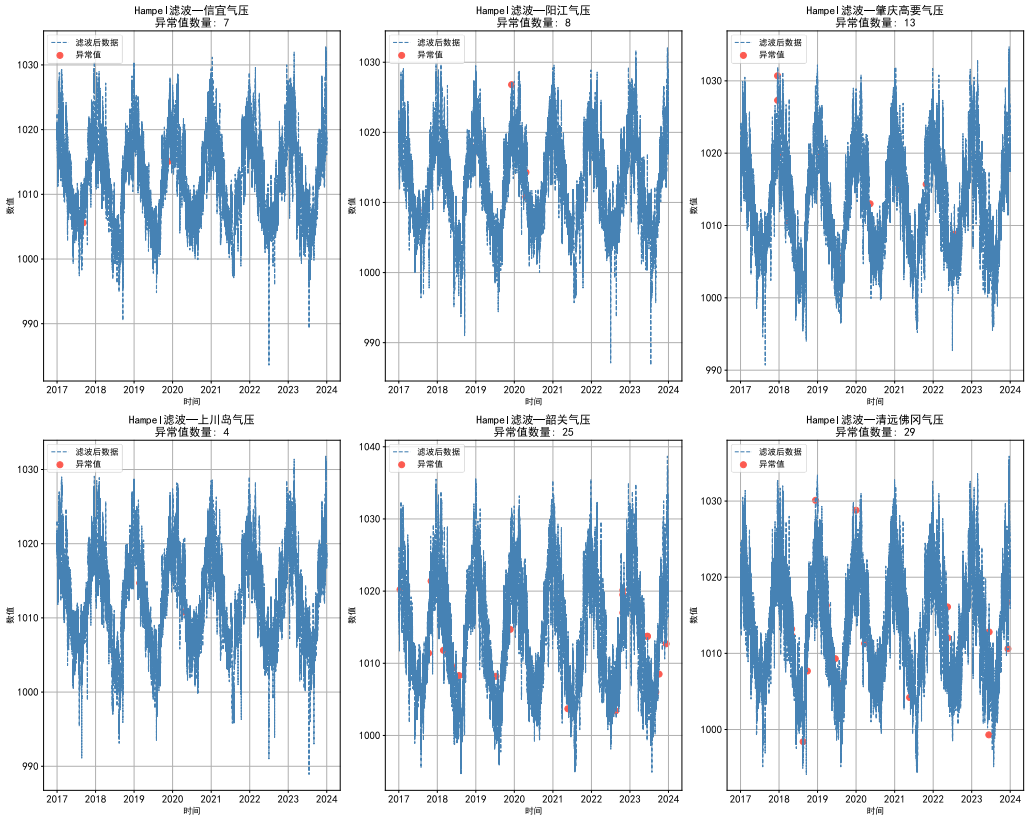


图 11 气压异常值处理

算法 1：汉普尔滤波器算法

```
输入：存在异常值的数据、窗口大小 L、阈值 k
输出：异常值处理后的数据

For 每个数据集
  For 站点：
    For 数据集点 y[i]：
      计算窗口 L 中 y[i] 的中位数(med)
      For 窗口 L 内的每个点 y[j]：
          计算绝对偏差 |y[j] - med|
      End
      计算窗口 L 内所有绝对偏差的中位数(MAD)
      If |y[i] - med| > k * MAD：
          y[i] = med
      Else：
          y[i] = y[i]
    End
  End
End
```

4. 数据标准化处理

为避免数据集中特征的取值范围不同，导致部分特征对结果影响较大，本次研究在进行气象预测前，对所用四个数据集分别进行标准化处理，如下式(12)。该步骤将有效消除特征的量纲差异性，提高模型的性能和鲁棒性。

$$X_{std} = \frac{X - \mu}{\sigma} \tag{12}$$

其中X_{std}是标准化后的数据，X是原始数据，μ、σ分别是原始数据的均值和标准差。

5. 实验环境

本次模拟实验使用的计算机配置为 Ubuntu20.04 系统、AMD EPYC-9754 18

vCPU、RTX4090D(24GB)GPU，使用的软件版本为 Python3.8、PyTorch 2.00、Cuda11.8。所有实验均采用 Amda 优化器(初始学习率为 1e-3)，迭代次数(epochs)为 200，批大小(batch_size)为 64，随机种子为 2024。(**注：所提模型并不需要高性能计算机，Intel i5-12500H CPU 个人电脑即可在 20s 内完成 1 轮的训练推理任务。采用高性能计算机的原因在于本次实验需要 7 个模型在四类数据集上进行性能测试。**)

6. 损失函数与模型评价指标

所提模型的损失函数设置为预测值与真实值的均方误差(MSE)如下式(13)，

$$\mathcal{L}(y,\hat{y}|\theta)=\frac{1}{k}\sum_{t=1}^{k}(y_t-\hat{y}_t)^2 \tag{13}$$

其中y_t为真实值，$\hat{y}_t$为预测值，k为训练样本数。

将均方根误差(RMSE)如下式(14)、平均绝对误差(MAE)如下式(15)作为检验模型性能的评价指标，用于后续实验工作。

$$\mathrm{RMSE}=\sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i-y_i)^2} \tag{14}$$

$$\mathrm{MAE}=\frac{1}{n}\sum_{i=1}^{n}\left|\hat{y}_i-y_i\right| \tag{15}$$

其中y_i为真实值，$\hat{y}_i$为预测值，n为测试样本数。

7. 对比实验与消融实验

① 对比试验

本次对比实验选取中长期多变量预测任务中，4 种具有代表性的方法，可分为基于循环神经网络架构的 LSTM、GRU；基于线性网络的 Dlinear[29]、基于

transformer 架构的 Informer。所提方法与这些方法的在温度、气压、风速、露点温度四类气象数据集上的对比结果如下**表 3**。（其中 1，7 等为各时间步，**加粗代表效果最佳，下划线标注表示效果次佳**）

表 3 对比实验结果

模型		TCBB(ours)		LSTM		GRU		Dlinear		Informer	
性能指标		MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
温度	1	**0.093**	**0.123**	0.140	0.188	0.151	0.202	0.173	0.245	0.186	0.245
	7	**0.085**	**0.112**	0.131	0.176	0.142	0.191	0.268	0.375	0.172	0.228
	13	**0.084**	**0.111**	0.128	0.173	0.139	0.187	0.350	0.471	0.166	0.221
	19	**0.084**	**0.111**	0.127	0.173	0.138	0.189	0.387	0.518	0.167	0.224
	25	**0.092**	**0.121**	0.128	0.174	0.139	0.190	0.403	0.534	0.186	0.248
	31	0.130	0.178	**0.128**	**0.174**	0.140	0.190	0.414	0.548	0.181	0.239
	AVG	**0.095**	**0.126**	0.130	0.176	0.142	0.192	0.333	0.449	0.176	0.234
		性能提升		27.37%	28.54%	33.10%	34.20%	71.53%	71.91%	46.31%	46.19%
气压	1	**0.057**	**0.080**	0.091	0.128	0.094	0.130	0.094	0.134	0.328	0.425
	7	**0.051**	**0.073**	0.077	0.110	0.081	0.115	0.213	0.292	0.351	0.457
	13	**0.051**	**0.072**	0.074	0.107	0.078	0.111	0.325	0.430	0.388	0.501
	19	**0.051**	**0.074**	0.074	0.108	0.077	0.112	0.391	0.503	0.413	0.526
	25	**0.054**	**0.076**	0.074	0.104	0.077	0.109	0.429	0.545	0.428	0.544
	31	**0.072**	**0.101**	0.074	0.106	0.078	0.110	0.447	0.568	0.436	0.556
	AVG	**0.056**	**0.079**	0.077	0.111	0.081	0.115	0.317	0.412	0.391	0.502
		性能提升		27.59%	28.21%	30.72%	30.71%	82.31%	80.74%	85.67%	84.18%
风速	1	**0.295**	**0.385**	0.420	0.552	0.433	0.570	0.660	0.888	0.739	0.962
	7	**0.269**	**0.347**	0.395	0.520	0.413	0.546	0.725	0.949	0.744	0.966
	13	**0.264**	**0.341**	0.390	0.514	0.410	0.541	0.760	0.991	0.753	0.983
	19	**0.266**	**0.343**	0.388	0.510	0.408	0.539	0.761	0.987	0.749	0.974
	25	**0.295**	**0.380**	0.388	0.511	0.408	0.540	0.768	0.998	0.754	0.983
	31	0.438	0.574	**0.388**	**0.510**	0.408	0.540	0.771	1.002	0.755	0.984
	AVG	**0.305**	**0.395**	0.395	0.520	0.413	0.546	0.741	0.969	0.749	0.975
		性能提升		22.88%	23.97%	26.33%	27.66%	58.90%	59.24%	59.35%	59.50%
露点温度	1	**0.082**	**0.111**	0.099	0.134	0.106	0.142	0.098	0.146	0.181	0.241
	7	**0.076**	**0.103**	0.092	0.123	0.098	0.131	0.240	0.359	0.181	0.244
	13	**0.075**	**0.102**	0.090	0.122	0.096	0.130	0.348	0.499	0.179	0.242
	19	**0.076**	**0.104**	0.090	0.122	0.096	0.131	0.394	0.558	0.184	0.251
	25	**0.080**	**0.108**	0.091	0.122	0.096	0.130	0.412	0.585	0.198	0.268
	31	0.099	0.134	**0.091**	**0.122**	0.097	0.132	0.431	0.607	0.190	0.256
	AVG	**0.081**	**0.110**	0.092	0.124	0.098	0.133	0.321	0.459	0.186	0.250
		性能提升		11.75%	11.14%	17.15%	16.83%	74.62%	75.96%	56.15%	55.93%

本次对比实验可以得出以下结论：①所提模型在各类数据集均表现最优，相较于其中表现较佳的 LSTM 也存在 11%的性能提升。②线性网络与 transformer 架构在此次实验中表现较差，循环神经网络占据优势，这或许是因为线性网络与 transformer 架构更适合超长期预测任务（依靠 512 步预测 96 步-336 步），这意味着依靠过去 64 天预测未来 12 天，甚至 42 天气象状态。然而此类任务不是本文重点研究目标，本次研究并未对其展开深入探讨。

② 消融实验

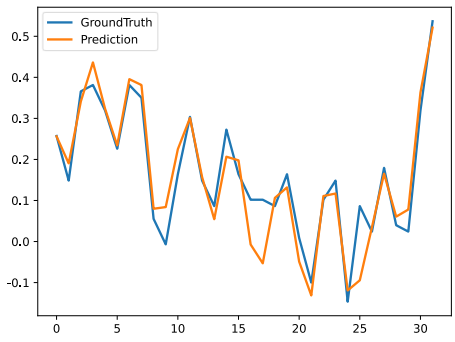
消融实验部分，选取双向长短期记忆网络 BiLSTM（拆除时域卷积与注意力模块）、时域卷积-双向长短期记忆网络 TCB（拆除注意力模块）与所提的混合模型（TCCB）在四类气象要素数据集上进行实验。消融实验结果如**表 4** 所示。

表 4 消融实验结果

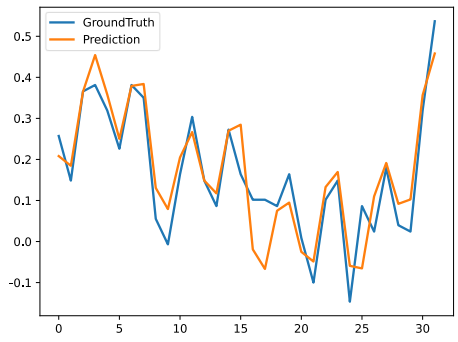
模型		TCBB(ours)		TCB		BiLSTM	
性能指标		MAE	RMSE	MAE	RMSE	MAE	RMSE
温度	1	**0.093**	**0.123**	0.105	0.139	0.106	0.141
	7	**0.085**	**0.112**	0.098	0.130	0.098	0.130
	13	**0.084**	**0.111**	0.097	0.128	0.096	0.127
	19	**0.084**	**0.111**	0.097	0.129	0.097	0.129
	25	**0.092**	**0.121**	0.106	0.140	0.107	0.142
	31	**0.130**	**0.178**	0.138	0.185	0.141	0.190
	AVG	**0.095**	**0.126**	0.107	0.142	0.108	0.143
		性能提升		11.39%	11.16%	11.94%	11.99%
气压	1	**0.057**	**0.080**	0.066	0.092	0.066	0.094
	7	**0.051**	**0.073**	0.058	0.084	0.059	0.085
	13	**0.051**	**0.072**	0.057	0.082	0.058	0.084
	19	**0.051**	**0.074**	0.058	0.084	0.059	0.087
	25	**0.054**	**0.076**	0.061	0.086	0.063	0.089
	31	**0.072**	**0.101**	0.079	0.110	0.081	0.113
	AVG	**0.056**	**0.079**	0.063	0.090	0.064	0.092
		性能提升		11.35%	11.52%	12.95%	13.77%
风速	1	0.295	0.385	**0.294**	**0.380**	0.329	0.431
	7	**0.269**	**0.347**	0.274	0.355	0.297	0.388
	13	**0.264**	**0.341**	0.271	0.350	0.292	0.380
	19	**0.266**	**0.343**	0.274	0.354	0.295	0.384
	25	**0.295**	**0.380**	0.304	0.393	0.322	0.420
	31	0.438	0.574	0.441	0.576	**0.434**	**0.570**
	AVG	**0.305**	**0.395**	0.310	0.401	0.328	0.429
		性能提升		1.67%	1.58%	7.21%	7.89%
露点温度	1	0.082	0.111	**0.078**	**0.105**	0.131	0.181
	7	0.076	0.103	**0.073**	**0.098**	0.126	0.175
	13	0.075	0.102	**0.072**	**0.097**	0.126	0.175
	19	0.076	0.104	**0.073**	**0.098**	0.129	0.178
	25	0.080	0.108	**0.077**	**0.104**	0.133	0.185
	31	0.099	0.134	**0.095**	**0.129**	0.145	0.200
	AVG	0.081	0.110	**0.078**	**0.105**	0.132	0.182
		性能提升		-4.27%	-4.91%	38.23%	39.49%

本次消融实验可以得出以下结论：①所提出的混合架构合理，时域卷积与注意力模块可为长短期记忆网络带来 11%-38%的性能提升。②注意力模块在温度、气压、风速数据集上均有明显作用。③注意力模块在露点温度数据集上作用不明显。本团队认为露点温度作为一种在饱和空气下，导致水蒸气凝结成露珠的温度，其变化幅度较低，信息含量稀少，不利于注意力模块发挥其强大作用。

为进一步展示所提模型的效果，下面提供各类模型在气压数据集上的表现，可明显看出所提模型总体而言拟合效果最佳，在捕获局部特征时展现优异的能力。这得益于通道空间注意力机制使时域卷积更专注于重要特征的提取，与双向长短期记忆神经网络具备根据上下文信息辅助获取中长期时序依赖的强大能力。对于 Informer 的不正常行为，本团队认为 transformer 架构对于该类数据存在“不适应”，Dlinear 团队同样证明 transformer 并不适合所有时序预测任务。**（蓝线代表真实值，橙线代表预测值）**

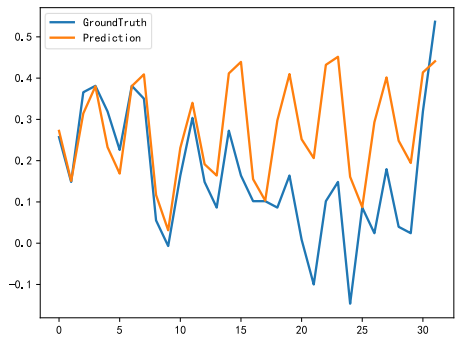


TCCB

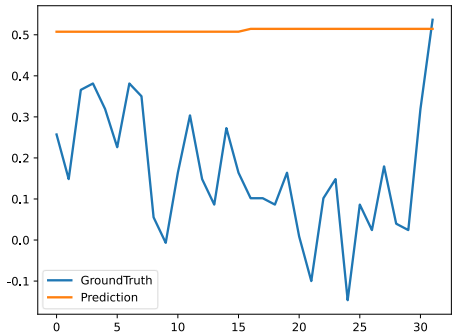


LSTM

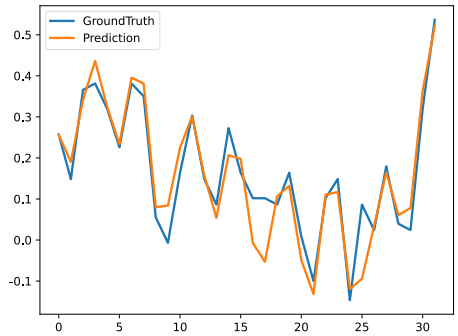
GRU



Dlinear

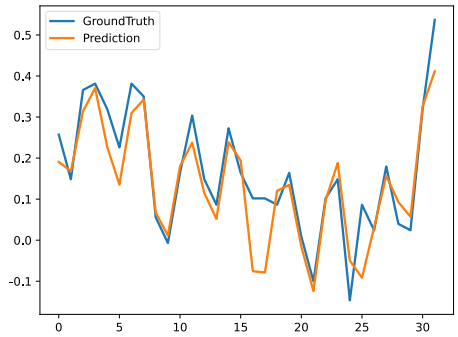


Informer



TCCB

TCB



Bilstm

四、 结论与探讨

（一） 模型的优势

针对中期多变量气象数据预测问题，本文深入探讨数值天气预报系统、统计方法及传统机器学习在此类问题上的不足，通过盘古等人工智能方法证明了拥有海量数据的当代，基于深度学习的人工智能方法在气象预测领域的优越性。为此，本研究针对中期、多站点两个关键词构建“基于时域卷积注意力优化的双向长短

期记忆神经网络”的混合架构，该架构充分发挥通道空间注意力机制对于捕获多变量、多时间步间重要特征的能力，显著提高后续时间卷积神经网络和双向长短期记忆神经网络的预测效果。同时，本次研究通过粤港澳 17 个气象站点数据（包含温度、气压、露点温度、风速）的实证分析，证明所提模型在绝大部分气象预测任务上具有强大的竞争力。

（二） 模型的不足及改进

本次研究总体完成度高，但仍存在以下不足：①本文通过多元预测的方法处理多站点信息，并未完全利用气象站点的空间地理信息（如经纬度，地势、海拔），这必然会导致模型丧失该部分数据带来的信息增益。②收集数据期间，本团队注意到部分气象站点的部分气象数据存在缺失严重情况（云量、1 小时降雨量缺失情况高达 80%以上），对于此类情况，常见的缺失数据插补方法难以奏效，本团队无奈忽略该部分数据，然而云量、1 小时降雨量属于重要气象要素，设计一种方法在数据缺失高达 80%情况下，依旧能用于预测具有一定意义。

为此，本团队提出以下改进：①对于进行时空联合预测，本团队在此给出一个粗略但可行的方案，设计一个以图卷积网络为主体的特征提取器，与所提模型组成形成新的并联架构，用于同时进行时空预测任务。②对于数据缺失异常严重的情况，本团队建议采用元学习的思想，将上述所提的并联架构作为元学习器，在完整度高的数据集上进行元训练学习，在完整度低的数据集上进行元测试学习。国内外已有相应学者在类似任务上证明所提方案具有一定的合理性[30][31]。

五、 参考文献

[1] WORLD METEOROLOGICAL ORGANZIATION,State of the Global Climate 2023,https://digitallibrary.un.org/record/4041156/files/1400365_EN.pdf

[2] 中国气象局,2023 年中国气候公报 https://www.cma.gov.cn/zfxxgk/gknr/qxbg/202402/t20240223_6084527.html

[3] 中国气象局,国家发展改革委,全国气象发展“十四五”规划 https://www.cma.gov.cn/zfxxgk/gknr/ghjh/202112/P020211209358186806802.pdf

[4] 黄小猛, 林岩銮, 熊巍, 等. 数值预报 AI 气象大模型国际发展动态研究[J]. 大气科学学报, 2024, 47(1): 46-54.

[5] Jaseena K U, Kovoor B C. Deterministic weather forecasting models based on intelligent predictors: A survey[J]. Journal of King Saud University-Computer and Information Sciences, 2022, 34(6): 3393-3412.

[6] Abbe C. The physical basis of long-range weather forecasts[J]. Monthly Weather Review, 1901, 29(12): 551-561.

[7] Bjerknes V. Das Problem der Wettervorhersage, betrachtet vom Standpunkte der Mechanik und der Physik[J]. Meteor. Z., 1904, 21: 1-7.

[8] Wang H, Zhang X Y, Wang P, et al. Chemistry-Weather Interacted Model System GRAPES_Meso5. 1/CUACE CW V1. 0: Development, Evaluation and Application in Better Haze/Fog Prediction in China[J]. Journal of Advances in Modeling Earth Systems, 2022, 14(12): e2022MS003222.

[9] Feng J, Huang X Y, Li Y. Improving surface wind speed forecasts using an offline surface multilayer model with optimal ground forcing[J]. Journal of Advances in Modeling Earth Systems, 2022, 14(10): e2022MS003072.

[10] 张珂铨. 人工智能算法在多时间尺度气象要素预测中的应用研究[D].兰州大学,2023.DOI:10.27204/d.cnki.glzhu.2022.003731.

[11] Farsi M, Hosahalli D, Manjunatha B R, et al. Parallel genetic algorithms for optimizing the SARIMA model for better forecasting of the NCDC weather data[J]. Alexandria Engineering Journal, 2021, 60(1): 1299-1316.

[12] 尹彤. 基于集成学习的武汉市空气质量分析及预测[D].中国石油大学(北京),2022.DOI:10.27643/d.cnki.gsybu.2022.001572.

[13] 孟建军,江相君,孟高阳,等.基于 IBES-XGBoost 的高速铁路沿线风速预测模型[J].灾害学,2024,39(01):57-63.

[14] Ren X, Li X, Ren K, et al. Deep learning-based weather prediction: a survey[J]. Big Data Research, 2021, 23: 100178.

[15] Zang H, Liu L, Sun L, et al. Short-term global horizontal irradiance forecasting based on a hybrid CNN-LSTM model with spatiotemporal correlations[J]. Renewable Energy, 2020, 160: 26-41.

[16] Karevan Z, Suykens J A K. Transductive LSTM for time-series prediction: An application to weather forecasting[J]. Neural Networks, 2020, 125: 1-9.

[17] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017, 30.

[18] Zhou H, Zhang S, Peng J, et al. Informer: Beyond efficient transformer for long sequence time-series forecasting[C]//Proceedings of the AAAI conference on artificial intelligence. 2021, 35(12): 11106-11115.

[19] Wu H, Zhou H, Long M, et al. Interpretable weather forecasting for worldwide stations with

a unified deep model[J]. Nature Machine Intelligence, 2023, 5(6): 602-611.
[20] Wu H, Xu J, Wang J, et al. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting[J]. Advances in neural information processing systems, 2021, 34: 22419-22430.
[21] Xu L, Zhang X, Du W, et al. Pentad-mean air temperature prediction using spatial autocorrelation and attention-based deep learning model[J]. Theoretical and Applied Climatology, 2023: 1-15.
[22] Bai S, Kolter J Z, Koltun V. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling[J]. arXiv preprint arXiv:1803.01271, 2018.
[23] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 770-778.
[24] Woo S, Park J, Lee J Y, et al. Cbam: Convolutional block attention module[C]//Proceedings of the European conference on computer vision (ECCV). 2018: 3-19.
[25] Meng, Anbo, et al. "An adaptive variational mode decomposition for wind power prediction using convolutional block attention deep learning network." Energy 282 (2023): 128945.
[26] Hochreiter S, Schmidhuber J. Long short-term memory[J]. Neural computation, 1997, 9(8): 1735-1780.
[27] Huang Z, Xu W, Yu K. Bidirectional LSTM-CRF models for sequence tagging[J]. arXiv preprint arXiv:1508.01991, 2015.
[28] Liang T, Chai C, Sun H, et al. Wind speed prediction based on multi-variable Capsnet-BILSTM-MOHHO for WPCCC[J]. Energy, 2022, 250: 123761.
[29] Zeng, Ailing, et al. "Are transformers effective for time series forecasting?." *Proceedings of the AAAI conference on artificial intelligence*. Vol. 37. No. 9. 2023.
[30] Lin H, Gao Z, Xu Y, et al. Conditional local convolution for spatio-temporal meteorological forecasting[C]//Proceedings of the AAAI conference on artificial intelligence. 2022, 36(7): 7470-7478.
[31] Lu B, Gan X, Zhang W, et al. Spatio-temporal graph few-shot learning with cross-city knowledge transfer[C]//Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2022: 1162-1172.

六、 附录

代码基于 pytorch 框架编写，详细阐释了所提模型的全构造过程，核心代码由七大 python 类模块构成。ChannelAttention、SpatialAttention、CBAMAttention 模块构成通道空间注意力机制。Chomp1d、TemporalBlock_CBAM、TCN_CBAM 模块构成结合注意力机制的时域卷积神经网络。TCN_CBAM_BILSTM 模块为总体模型。该代码附上相应注释，供评委老师检查。

```python
import torch
import torch.nn as nn
from torch.nn.utils import weight_norm


class ChannelAttention(nn.Module):
    def __init__(self, channel, reduction=8):
        super().__init__()
        self.maxpool = nn.AdaptiveMaxPool1d(1)
        self.avgpool = nn.AdaptiveAvgPool1d(1)
        self.se = nn.Sequential(
            nn.Conv1d(channel, channel // reduction, 1, bias=False),
            nn.ReLU(inplace=True),
            nn.Conv1d(channel // reduction, channel, 1, bias=False)
        )
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        max_result = self.maxpool(x)
        avg_result = self.avgpool(x)
        #print("1",max_result.shape, avg_result.shape)
        max_out = self.se(max_result)
        #print("3",max_out.shape)
        avg_out = self.se(avg_result)
        output = self.sigmoid(max_out + avg_out)
        print(output.shape)
        return output


class SpatialAttention(nn.Module):
    def __init__(self, kernel_size_1=7):
        super().__init__()
        self.conv = nn.Conv1d(2, 1,
kernel_size=kernel_size_1,padding=kernel_size_1 // 2)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        max_result, _ = torch.max(x, dim=1, keepdim=True)
        #print("ma",max_result.shape)
        avg_result = torch.mean(x, dim=1, keepdim=True)
        result = torch.cat([max_result, avg_result], 1)
        #print("sh",result.shape)
        output = self.conv(result)
        #print("out",output.shape)
```

```
        output = self.sigmoid(output)
        return output


class CBAMAttention(nn.Module):
    def __init__(self, channel=32, reduction=1, kernel_size_1=7):
        super().__init__()
        self.ca = ChannelAttention(channel=channel,
reduction=reduction)
        self.sa = SpatialAttention(kernel_size_1=kernel_size_1)

    def forward(self, x):
        b, c, _= x.size()
        out = x * self.ca(x)
        #print(out.shape)
        out = out * self.sa(out)
        return out


"""
这个是 CBAM(通道空间注意力机制)


  |最大池化层(Maxpool)                                        covlayer
（卷积）      ||     ||
----多层感知机（MLP）  + (sigmoid 激活函数)  ----  池化（压缩）-----------
----------  || --- ||-------| |
  |平均池化层(Avgpool)
||     ||

具体可查看通道空间注意力机制图
"""

class Chomp1d(nn.Module):
    def __init__(self, chomp_size):
        super(Chomp1d, self).__init__()
        self.chomp_size = chomp_size

    def forward(self, x):
        """
        裁剪模块，目的是保持输入与输出的格式一致
        """
        return x[:, :, :-self.chomp_size].contiguous()
#################################################################
```

```
"""
Dilated Causal Conv  因果空洞卷积
WeghtNorm            权重归一化
ReLU                 ReLU 激活函数
Dmpout               正则化
1x1 Conv             1*1 卷积
"""
class TemporalBlock_CBAM(nn.Module):
    def __init__(self, n_inputs, n_outputs, kernel_size, stride,
dilation, padding, dropout=0.2):
        """
        相当于一个 Residual block

        n_inputs: int, 输入通道数
        n_outputs: int, 输出通道数
        kernel_size: int, 卷积核尺寸
        stride: int, 步长, 一般为 1
        dilation: int, 膨胀系数
        padding: int, 填充系数
        dropout: float, dropout 比率
        """
        super(TemporalBlock_CBAM, self).__init__()
        self.conv1 = weight_norm(nn.Conv1d(n_inputs, n_outputs,
kernel_size,
                                       stride=stride, padding=padding,
dilation=dilation))
        # 输出的张量为 (Batch, input_channel, seq_len + padding)
        self.chomp1 = Chomp1d(padding)  # 裁剪 padding
        self.relu1 = nn.ReLU()
        self.dropout1 = nn.Dropout(dropout)
        self.cbam1 = CBAMAttention()    # 注意力机制
        self.conv2 = weight_norm(nn.Conv1d(n_outputs, n_outputs,
kernel_size,
                                       stride=stride, padding=padding,
dilation=dilation))
        self.chomp2 = Chomp1d(padding)  # 裁剪 padding
        self.relu2 = nn.ReLU()
        self.dropout2 = nn.Dropout(dropout)
        self.cbam2 = CBAMAttention()    # 注意力机制

        self.net = nn.Sequential(self.conv1, self.chomp1,
self.relu1, self.dropout1,self.cbam1,
                                 self.conv2, self.chomp2, self.relu2,
self.dropout2,self.cbam2,)
```

```
        self.downsample = nn.Conv1d(n_inputs, n_outputs, 1) if
n_inputs != n_outputs else None
        self.relu = nn.ReLU()
        self.init_weights()

    def init_weights(self):
        """
        初始化参数
        """
        self.conv1.weight.data.normal_(0, 0.01)
        self.conv2.weight.data.normal_(0, 0.01)
        if self.downsample is not None:
            self.downsample.weight.data.normal_(0, 0.01)

    def forward(self, x):
        """
        x: 的尺寸为 (Batch, input_channel, seq_len)
        """
        out = self.net(x)
        res = x if self.downsample is None else self.downsample(x)
        return self.relu(out + res)

#################################################################
"""
通道空间时域卷积神经网络
"""

class TCN_CBAM(nn.Module):
    def __init__(self, num_inputs, num_channels, kernel_size=2,
dropout=0.2):
        """

        num_inputs: int,  输入通道数
        num_channels: list，每层的 hidden_channel 数，例如[32,32,32]表示
有 3 个隐层，每层 hidden_channel 数为 32
        kernel_size: int, 卷积核尺寸
        dropout: float, drop_out 比率

        """
        super(TCN_CBAM, self).__init__()
        layers = []
        num_levels = len(num_channels)
        for i in range(num_levels):
            dilation_size = 2 ** i  # 膨胀系数  扩散因子
```

```
            in_channels = num_inputs if i == 0 else num_channels[i - 1]  # 确定输入通道数
            out_channels = num_channels[i]  # 确定输出通道数
            layers += [TemporalBlock_CBAM(in_channels, out_channels, kernel_size, stride=1, dilation=dilation_size,
                                          padding=(kernel_size - 1) * dilation_size, dropout=dropout)]

        self.network = nn.Sequential(*layers)

    def forward(self, x):

        """
         x: 的尺寸为 (Batch, input_channel, seq_len)
        """

        return self.network(x)


#####################################################################
"""
总体模型
"""
class TCN_CBAM_BILSTM(nn.Module):
    def __init__(self):
        super(TCN_CBAM_BILSTM, self).__init__()
        self.tcn = TCN_CBAM(num_inputs=17, num_channels=[32, 32, 32])
        self.bilstm = nn.LSTM(input_size=32, hidden_size=64,num_layers=2, batch_first=True,bidirectional=True)
        self.fc = nn.Linear(64*2,17)
        self.pre_len = 32

    def __str__(self):
        return "TCN_CBAM_BILSTM"

    def forward(self, x):
        x = x.permute(0,2,1)     # batch_size,seq_length,in_feature -- batch_size,in_feature,seq_length
        x = self.tcn(x)
        x = x.permute(0, 2, 1)   # batch_size,hidden_dim,seq_length--batch_size,seq_length,hidden_dim
        h0 = torch.zeros(2 * 2, x.size(0), 64).to(x.device)
        c0 = torch.zeros(2 * 2, x.size(0), 64).to(x.device)
```

```
        #print("x_shape",x.shape)
        x = self.fc(x)        # batch_size,seq_length,hidden_dim--
batch_size,seq_length,out_dim     [32,64,17]
        #print("x_shape",x.shape)
        x = x[:,-self.pre_len:,:]
        return x
```